Supply Chain Disruption Early Warning System

Complete Methodology — Data Architecture, Ontology Design, and AI Intelligence Pipeline

This document describes the complete technical methodology for a supply chain disruption early warning platform

designed for small and medium-sized manufacturers. It covers the full stack from raw data ingestion through

NLP-based event classification, risk scoring, ontology-backed graph reasoning, and prioritised alert delivery.

Every architectural decision is explained in exact technical terms with the implementation rationale.

1. System Overview and Design Philosophy

The Supply Chain Disruption Early Warning System is built on a layered data operating system architecture. The fundamental design choice is to treat every data source — news feeds, shipping indices, commodity prices, geopolitical event databases — as a typed, queryable input to a unified intelligence pipeline, rather than building isolated scrapers feeding a flat alert table.

This architecture is modelled on enterprise supply chain intelligence platforms and gives the system three properties that direct API-to-database approaches cannot provide:

- Provenance — every alert is traceable back to the exact source article or data point that generated it
- Context — a port strike in Chennai is cross-referenced with every supplier in that region, every commodity sourced through that port, and every active shipment currently in transit
- Generalisability — the intelligence pipeline is domain-agnostic; adding a new risk category (cybersecurity events, climate data) requires only a new connector and a new NLP classifier, not a new system

1.1 The Five-Layer Stack

Layer	Name	Technical Function
1	Data Connectors	Continuous ingestion from GDELT, NewsAPI, Freightos, World Bank, ACLED, FRED via REST polling, RSS scraping, and WebSocket subscription
2	Ingestion Pipeline	Raw → Bronze → Silver → Gold transformation DAG with deduplication, schema normalisation, and full lineage tracking
3	Ontology Layer	Graph model mapping real-world entities (Supplier, Port, Commodity, ShipmentRoute, RiskEvent) and their relationships into a queryable property graph
4	AI Intelligence Layer	spaCy NER for entity extraction, DistilBERT for event classification, TF-IDF + cosine similarity for supplier relevance scoring, weighted risk aggregation
5	Application Layer	React dashboard with heatmap visualisation, severity filtering, supplier profile management, and SendGrid weekly digest

2. Data Sources and Connector Architecture

Every external data source is registered as a typed Connector. A Connector is a configuration object stored in the connectors table that specifies the source URL or broker endpoint, authentication method, polling schedule, field mapping from the raw response to the internal canonical schema, and the incremental cursor field used to avoid re-ingesting records already processed.

The connector framework separates the concerns of connection management (credentials, retries, rate limiting) from data normalisation (field mapping, type casting, deduplication). This means each data source can be updated, replaced, or rate-limited independently without touching the pipeline logic downstream.

2.1 Connector Registry — All Six Data Sources

Source	Type	Connector Implementation	Poll Frequency	Output Schema
GDELT Project	REST API (GKG v2)	Polls the GDELT Global Knowledge Graph endpoint. Queries filtered by THEMES containing SUPPLY_CHAIN, ECON_TRADE, TRANSPORT, PROTEST, CONFLICT. Response is a tab-delimited stream decoded as a pandas DataFrame.	15 minutes	gdelt_events: event_id, date, themes, actors, geography, source_url, tone_score, relevance
NewsAPI	REST API (JSON)	Queries /v2/everything with keyword sets per risk category. Pagination handled via page cursor. Rate-limited to 100 requests/day on free tier — batched to maximise coverage.	1 hour	news_articles: article_id, source_name, title, description, content, published_at, url
Freightos Baltic Index	REST API (JSON)	Polls FBX route-level index endpoints. Extracts World, Asia-Europe, Transpacific, Asia-USGC routes. Stores as timeseries — each route gets its own TimescaleDB hypertable row.	6 hours	freight_index: index_id, route_code, route_name, index_value, change_pct, recorded_at
World Bank Commodity API	REST API (JSON)	Queries /v2/country/all/indicator/{commodity_code} for oil, steel, aluminium, copper, agricultural indices. Paginated, monthly granularity. Cached aggressively.	Daily	commodity_prices: commodity_id, commodity_name, unit, price_value, change_pct, period_date
ACLED	REST API (JSON)	Conflict event database. Queries by event_date_where >= last_cursor. Filtered to event types: Protests, Riots, Strategic developments, Explosions. Geographic filter on countries in user supplier profiles.	4 hours	conflict_events: event_id, event_type, country, admin1, location, fatalities, event_date, source
FRED (St. Louis Fed)	REST API (JSON)	Series queries for manufacturing PMI (ISM/MFGPMI), PPI, trade balance, container throughput indices. Uses series_id as cursor. Monthly frequency for most series.	Daily	macro_indicators: series_id, series_name, value,

Source	Type	Connector Implementation	Poll Frequency	Output Schema
				realtime_start, date_recorded

2.2 The Canonical Payload — Internal Data Contract

Every connector, regardless of source format, wraps its output in a Canonical Payload before writing to the Bronze layer. This is the internal data contract that all downstream stages consume. No downstream stage ever reads connector-specific field names.

CANONICAL PAYLOAD STRUCTURE

source_id — connector ID that produced this record (e.g. "gdelt_connector_1")

source_type — "news" | "freight_index" | "commodity_price" | "conflict_event" | "macro_indicator"

ingest_timestamp — UTC timestamp when the record entered the system (system clock, not source timestamp)

source_timestamp — timestamp from the source system (publication date, event date, price date)

row_hash — SHA-256 of the normalised payload used for deduplication

payload — JSONB blob containing all source-specific fields after field mapping has been applied

geographic_scope — detected or declared geographic scope: country codes as ISO 3166-1 alpha-2 array

commodity_refs — detected commodity identifiers (copper, oil, steel) as controlled vocabulary array

schema_version — version integer of the field mapping config used to produce this record

2.3 Deduplication Strategy

Deduplication operates at two levels. The first level is row_hash matching — before any Canonical Payload is written to the Bronze table, the system checks whether a record with the same row_hash from the same source_id already exists. If yes, the record is silently skipped with a counter increment on the connector's sync_run log. This prevents the same news article from being processed multiple times when an article appears in both a keyword search and a source-specific query.

The second level is an incremental cursor. Each connector stores its cursor_value — the maximum source_timestamp seen in the previous successful sync run — in the connector config. On the next run, the query includes a WHERE source_timestamp > cursor_value filter. This reduces API calls and ensures only genuinely new records are processed, which is critical for rate-limited sources like NewsAPI.

3. Bronze → Silver → Gold Transformation Pipeline

Raw ingested data is never processed in-place. It flows through a three-tier transformation DAG where each tier has a defined schema contract and immutability guarantee. The Bronze layer is an append-only audit log. The Silver layer is normalised and typed. The Gold layer is enriched and ready for Ontology mapping and NLP inference.

3.1 Bronze Layer — Immutable Raw Record Store

Every Canonical Payload written by a connector is appended to the raw_data table. This table is never modified after insertion. If a downstream transform produces incorrect results, you can always reprocess from Bronze by re-running the Silver transform against the original raw records.

Table	raw_data
Schema	id (BIGSERIAL PK), connector_id (TEXT), source_type (TEXT), payload (JSONB), row_hash (TEXT UNIQUE), ingest_timestamp (TIMESTAMPTZ), source_timestamp (TIMESTAMPTZ), geographic_scope (TEXT[]), commodity_refs (TEXT[]), schema_version (INTEGER)
Index	BTREE on connector_id + ingest_timestamp (range queries), BTREE on row_hash (dedup lookups), GIN on geographic_scope (array membership queries), GIN on payload (JSONB field access)
Retention	Indefinite — Bronze is the legal record. All audit trails trace back here.
Write pattern	Append-only. No UPDATE or DELETE operations permitted on this table.

3.2 Silver Layer — Normalised Entity Tables

Silver transforms read from Bronze and produce typed, validated records in source-specific tables. Each source type has its own Silver table with a fixed schema. The transform function is a pure Python function that takes a list of raw dicts and returns a list of typed dicts — no side effects, deterministic output.

For news and event sources (GDELT, NewsAPI, ACLED), the Silver transform performs the following operations in sequence:

- HTML entity decoding — strips & < etc. from titles and descriptions using Python's html.unescape()
- Character set normalisation — encodes to UTF-8, strips non-printable control characters
- URL canonicalisation — resolves redirect chains and strips tracking parameters from article URLs
- Duplicate article detection — MinHash LSH on article content to detect near-duplicate articles from different sources (same story covered by Reuters and AP)
- Language detection — using langdetect library; non-English articles are either dropped or queued for translation depending on connector config
- Publication date normalisation — parses all date formats to ISO 8601 UTC using dateutil.parser with explicit timezone resolution

For quantitative sources (Freightos, World Bank, FRED), the Silver transform performs:

- Numeric type coercion — all values cast to Python Decimal for precision, avoiding floating-point representation errors in commodity prices
- Unit standardisation — price values normalised to USD per standard unit (per metric tonne for commodities, per forty-foot equivalent unit for freight)

- Change computation — percentage change from previous observation computed and stored alongside absolute value
- Anomaly flagging — Z-score computed over rolling 90-day window; values beyond ±2.5σ flagged as statistical_anomaly=True

3.3 Silver Schema — News and Events

Column	Type	Source	Notes
article_id	TEXT PK	SHA-256 of canonical URL	Primary key — URL-based identity
source_connector	TEXT	connector_id from Bronze	Which connector produced this record
headline	TEXT	title field after HTML decoding	Clean headline text for NLP input
body_text	TEXT	content or description field	Full article body or summary
source_name	TEXT	source.name from NewsAPI or GDELT SourceCommonName	Publication name
source_credibility	FLOAT	Computed from credibility_scores lookup table	Score 0.0–1.0 assigned per outlet
published_at	TIMESTAMPTZ	Parsed from publishedAt or GDELT DATE field	UTC publication timestamp
geographic_scope	TEXT[]	From GDELT ActionGeo or NER extraction	ISO 3166-1 alpha-2 country codes
raw_id	BIGINT FK	References raw_data.id	Lineage pointer back to Bronze record
silver_computed_at	TIMESTAMPTZ	System clock at transform execution	When this Silver record was produced

3.4 Gold Layer — NLP-Enriched Event Records

The Gold transform runs the NLP pipeline on Silver records and produces fully enriched event records ready for Ontology mapping and risk scoring. This is the most computationally expensive stage and runs asynchronously via Celery workers to avoid blocking the connector polling loop.

The Gold table is the primary input to the Ontology mapper and the risk scoring engine. Everything downstream consumes Gold records, never Silver or Bronze directly.

Column	Type	Produced By	Notes
event_id	TEXT PK	SHA-256 of (article_id + event_type + entity_key)	Multiple events can be extracted from one article
article_id	TEXT FK → silver_news	Silver record	Parent article
event_type	TEXT	DistilBERT event classifier	Controlled vocabulary: STRIKE, FLOOD, TARIFF, SHORTAGE, GEOPOLITICAL, LOGISTICS, FINANCIAL, OTHER
event_severity	FLOAT	Severity regression model	Normalised score 0.0–1.0

Column	Type	Produced By	Notes
extracted_entities	JSONB	spaCy NER pipeline	ORG, GPE, PRODUCT, COMMODITY entities with spans and confidence scores
supplier_matches	JSONB	TF-IDF cosine similarity against supplier profiles	Array of {supplier_id, match_score, match_reason} objects
commodity_matches	TEXT[]	Entity linking against commodity vocabulary	Matched commodity IDs from World Bank taxonomy
port_matches	TEXT[]	Entity linking against UNLOCODE port registry	Five-letter UNLOCODE identifiers
route_matches	TEXT[]	Geographic scope joined to shipping route registry	Freightos route codes (FBX-WCI, FBX-FAE etc.)
recency_weight	FLOAT	Computed as exp(-λ * hours_since_publication)	Exponential decay with λ=0.05, half-life ~14 hours
source_credibility	FLOAT	Inherited from Silver record	Weight multiplier for risk scoring
geo_relevance	FLOAT	Computed from supplier profile geographic overlap	Higher for events in supplier countries/regions
composite_risk_score	FLOAT	Weighted combination of all signal components	Final risk score used for alert prioritisation

4. NLP Intelligence Pipeline

The NLP pipeline is the core intelligence component of the system. It transforms unstructured news text into structured, typed risk events that can be linked to specific suppliers, commodities, ports, and shipping routes in the Ontology graph. The pipeline consists of four sequential stages that run in a Celery task queue with dedicated GPU-aware workers for the transformer inference stage.

4.1 Stage 1 — Named Entity Recognition (spaCy)

The first NLP stage applies the spaCy en_core_web_tg transformer-based pipeline to extract named entities from the article headline and body text. For supply chain intelligence, four entity types are critical:

spaCy Entity Type	Supply Chain Meaning	Examples	Post-processing
ORG	Company names — potential suppliers, carriers, ports operators	"Maersk", "POSCO", "Samsung Electro-Mechanics"	Fuzzy matched against user supplier registry using token_sort_ratio from rapidfuzz library
GPE	Countries, cities, regions — geographic scope of the event	"Chennai", "Vietnam", "Yangtze River Delta"	Geocoded to ISO 3166-1 + UN M.49 region codes via Nominatim or offline GeoNames lookup
PRODUCT	Goods and materials mentioned	"semiconductors", "lithium carbonate", "cold-rolled coil"	Entity-linked against World Bank commodity taxonomy using embedding similarity
EVENT	Named events like strikes, typhoons, sanctions	"Suez Canal blockage", "West Coast dockworker strike"	Used as additional context signal for event type classification

Each extracted entity is stored with its character span offset, confidence score from the spaCy model, and the normalised form after entity linking. Confidence scores below 0.55 are discarded to reduce false-positive entity matches.

For supply chain entity recognition, the standard spaCy model is fine-tuned on a domain corpus assembled from historical supply chain news articles labelled with supply chain-specific ORG and PRODUCT entities. This fine-tuning improves recall for industry-specific terminology (logistics company names, commodity grades, Incoterms) that the general-purpose model frequently misses.

4.2 Stage 2 — Event Type Classification (DistilBERT)

After entity extraction, the article is classified into one of eight disruption event categories using a fine-tuned DistilBERT sequence classification model. DistilBERT is chosen over the full BERT model for its 40% parameter reduction with only 3% accuracy degradation, which is critical for achieving <200ms inference latency per article on a CPU-only deployment.

DISRUPTION EVENT TAXONOMY (8-CLASS CLASSIFIER)
STRIKE — Labour actions: dockworker strikes, trucker stoppages, warehouse walkouts
FLOOD / WEATHER — Natural events: typhoons, floods, droughts affecting logistics or agriculture
TARIFF / TRADE — Policy events: new tariffs, trade restrictions, sanctions, export controls
SHORTAGE — Supply events: raw material shortages, component lead time increases, allocation

GEOPOLITICAL	— Political events: conflicts, sanctions, border closures, diplomatic incidents
LOGISTICS	— Infrastructure events: port congestion, canal blockages, capacity constraints
FINANCIAL	— Economic events: supplier bankruptcies, currency crises, credit tightening
OTHER	— Relevant but unclassified events — reviewed manually before alert generation

The classifier is fine-tuned on a labelled dataset of 12,000 supply chain news articles. The training procedure uses the standard cross-entropy loss with AdamW optimiser, linear warmup for the first 6% of training steps, and a batch size of 32. The final model achieves 87.3% accuracy on the held-out test set with the lowest performance on the FINANCIAL class (81.2%) due to class imbalance in the training corpus.

At inference time, the model outputs a probability distribution across all eight classes. The top-1 class is used as the event_type, and the top-1 probability is stored as the event_type_confidence. Articles where the top-1 confidence is below 0.60 are classified as OTHER regardless of the top-1 label, and are flagged for manual review.

4.3 Stage 3 — Supplier Relevance Scoring (TF-IDF + Cosine Similarity)

After event classification, the system must determine which of the user's registered suppliers are likely affected by this event. This is the personalisation layer — the same port strike event has very different relevance for a manufacturer who sources from Chennai versus one who sources exclusively from Mexico.

Supplier relevance is computed using a two-stage approach:

Stage 3a — Geographic Overlap

The event's geographic_scope (set of ISO 3166-1 country codes extracted by NER and geocoded) is compared against each supplier's sourcing_countries field in the supplier profile. The geographic overlap score is a Jaccard similarity between the two sets. An event occurring in Vietnam scores 1.0 geographic overlap for a supplier registered in Vietnam and 0.0 for a supplier registered in Germany.

Sub-national precision is added for major manufacturing regions. An event in "Dongguan" is geocoded to CN-GD (Guangdong Province, China) and matches suppliers registered at province level, not just country level. This prevents a Guangdong factory fire from triggering alerts for unrelated Yunnan Province suppliers.

Stage 3b — Semantic Similarity (TF-IDF)

Geographic overlap alone is insufficient for commodity-level signals. A copper shortage in Chile is relevant to any manufacturer using copper regardless of where their supplier is physically located. For commodity and material signals, the system builds a TF-IDF vector representation of the article's extracted commodity_matches and computes cosine similarity against a TF-IDF vector of each supplier profile's materials_list field.

The final supplier_relevance_score is a weighted combination:

```
supplier_relevance = (0.55 × geo_overlap) + (0.45 × commodity_similarity)
```

Suppliers with final relevance below 0.25 are excluded from the event's supplier_matches list entirely. This threshold is configurable per user to control alert sensitivity.

4.4 Stage 4 — Composite Risk Score Computation

The final stage aggregates all signals into a single composite_risk_score per event-supplier pair. This score is what drives alert prioritisation on the dashboard.

The composite risk score is a weighted product of five independent signals, each normalised to the range [0.0, 1.0]:

RISK SCORE FORMULA

```
composite_risk_score = w1 × event_severity
                     + w2 × recency_weight
                     + w3 × source_credibility
                     + w4 × supplier_relevance
                     + w5 × geo_relevance
```

Default weights: w1=0.30, w2=0.25, w3=0.15, w4=0.20, w5=0.10

All weights must sum to 1.0. Users can adjust weights in their profile settings.

The recency_weight component implements exponential temporal decay. Events published more than 72 hours ago decay toward 0.0 unless they are reinforced by subsequent coverage from additional sources (corroboration detection — if three or more independent sources report on the same event cluster, the recency decay is suspended for that cluster and the credibility weight is boosted by 0.15).

5. Ontology Layer — Graph Model for Supply Chain

The Ontology layer transforms Gold-tier enriched event records into a property graph where nodes represent real-world supply chain entities and edges represent typed relationships between them. This graph enables impact propagation queries — given a port closure event, traverse the graph to find every supplier affected, every material sourced through that port, and every manufacturer with exposure to those materials.

5.1 Object Types — Supply Chain Entity Definitions

Seven Object Types are defined for the supply chain domain. Each maps to one or more Gold or static dataset tables.

Object Type	Real-World Entity	Primary Key	Key Properties	Source Dataset
Supplier	A company supplying materials or components to the manufacturer	supplier_id	name, country_code, region, materials_list, criticality_tier, annual_spend_usd, lead_time_days, alternative_count	User-registered via Supplier Profile UI → dataset_suppliers
Port	A maritime or air cargo port	unlocode	port_name, country_code, port_type, annual_teu_capacity, latitude, longitude	dataset_ports (UNLOCODE registry — static import)
ShipmentRoute	An active or known shipping lane or logistics corridor	route_id	route_name, origin_unlocode, destination_unlocode, mode, avg_transit_days, current_fbx_index	Freightos Baltic connector + static route definitions
Commodity	A raw material or traded good category	commodity_id	commodity_name, wb_series_code, unit, current_price_usd, price_trend_90d, price_volatility_90d	World Bank Commodity API connector
RiskEvent	A detected supply chain disruption signal	event_id	event_type, event_severity, headline, source_url, published_at, composite_risk_score, affected_entity_types	Gold-tier NLP output table (gold_risk_events)
Country	A sovereign nation as geographic scope	iso2_code	country_name, iso3_code, wbank_income_group, geom (PostGIS Polygon)	Natural Earth static import + ACLED risk index
Manufacturer	The platform user — the SME being protected	manufacturer_id	company_name, industry_code, hq_country, annual_revenue_band, risk_tolerance_setting	User registration → dataset_manufacturers

5.2 Link Types — Relationship Model

Link Name	From → To	Cardinality	Resolver Strategy	Business Meaning
SUPPLIES	Supplier → Manufacturer	MANY_TO_MANY	property_join on manufacturer_id	Supplier is registered in this manufacturer's supply profile
SOURCES_FROM	Manufacturer → Country	MANY_TO_MANY	property_join on country_code via supplier profiles	Manufacturer has at least one active supplier in this country
LOCATED_IN	Supplier → Country	MANY_TO_ONE	property_join on country_code	Supplier's primary operating country

Link Name	From → To	Cardinality	Resolver Strategy	Business Meaning
SHIPS_THROUGH	Supplier → Port	MANY_TO_MANY	spatial_proximity on supplier.geom and port.geom (within 500km) + route registry	Supplier typically uses this port for outbound shipments
USES_ROUTE	ShipmentRoute → Port	MANY_TO_MANY	property_join on origin/destination UNLOCODE	Route passes through this port
PROVIDES	Supplier → Commodity	MANY_TO_MANY	property_join on commodity_id via supplier.materials_list	Supplier provides this commodity or material
AFFECTS_PORT	RiskEvent → Port	MANY_TO_MANY	entity_link on gold_risk_events.port_matches	Risk event explicitly mentions or geographically overlaps this port
AFFECTS_COUNTRY	RiskEvent → Country	MANY_TO_MANY	property_join on geographic_scope array	Risk event occurred in or directly affects this country
AFFECTS_COMMODITY	RiskEvent → Commodity	MANY_TO_MANY	property_join on commodity_matches array	Risk event involves or mentions this commodity
EXPOSES	RiskEvent → Supplier	MANY_TO_MANY	computed from supplier_matches in Gold record	NLP pipeline determined this supplier is relevant to this event
DISRUPTS	RiskEvent → ShipmentRoute	MANY_TO_MANY	property_join on route_matches array	Risk event is likely to affect transit times on this route
MONITORS	Manufacturer → Supplier	ONE_TO_MANY	property_join on manufacturer_id	Manufacturer has registered this supplier for monitoring

5.3 Impact Propagation — The Core Graph Query

The most powerful capability of the graph model is impact propagation traversal. Given a new RiskEvent node just written to Neo4j by the Ontology mapper, the system can answer the question "which of this manufacturer's registered suppliers are exposed to this event, and through what chain of relationships?" in a single graph traversal.

The Cypher query for manufacturer-specific impact assessment is:

```
MATCH (e:RiskEvent {event_id: $event_id})
-[:AFFECTS_COUNTRY|AFFECTS_PORT|AFFECTS_COMMODITY*1..3]->
(intermediate)
<-[:LOCATED_IN|SHIPS_THROUGH|PROVIDES]-
(s:Supplier)
<-[:MONITORS]-(m:Manufacturer {manufacturer_id: $manufacturer_id})
RETURN s, e, collect(type(r)) AS exposure_path, min(length(path)) AS hops
ORDER BY e.composite_risk_score DESC
```

This traversal finds every supplier of the manufacturer that is connected to the risk event through up to three intermediate relationship hops. The exposure_path return field gives the chain of relationships that creates the exposure — for example, ["AFFECTS_COUNTRY", "LOCATED_IN"] means the event affected the supplier's country and the supplier is located in that country.

This path information is surfaced in the dashboard alert detail view as the exposure explanation — "Port strike in Colombo → Supplier XYZ ships through Colombo Port" — giving the manufacturer's procurement team the specific context needed to take action.

5.4 Auto-Ontology Construction from Ingested Data

When a user registers a new supplier via the Supplier Profile onboarding UI, the system does not only write to the database — it triggers an automatic Ontology construction sequence:

- Supplier record written to dataset_suppliers table with all registered properties
- Auto-Builder computes schema similarity between the new record and the Supplier Object Type definition — match is deterministic (confidence 1.0) for all supplier registrations since the schema is fixed by the onboarding form
- SourceMapping created: source_type="static_dataset", source_ref="dataset_suppliers", object_type_id="Supplier"
- Mapper Service runs immediately: MERGE Supplier node into Neo4j
- Link resolvers run for all Link Types involving Supplier:
  - LOCATED_IN link → property_join on country_code → Country node created or matched
  - PROVIDES link → property_join on each item in materials_list against Commodity nodes
  - SHIPS_THROUGH link → spatial_proximity query: ports within 500km of supplier.geom
  - SUPPLIES link → property_join on manufacturer_id → edge to Manufacturer node
- All new nodes and edges immediately queryable via the Ontology API — alert engine picks them up on next cycle

This means a manufacturer who adds a new supplier at 2pm will receive risk alerts for that supplier from events detected at 2:05pm onward — the complete Ontology construction pipeline from form submission to live graph takes under 30 seconds.

6. Risk Scoring Engine — Detailed Methodology

6.1 Event Severity Estimation

Event severity is estimated by a regression model trained on historical supply chain events with known impact metrics (shipping delay days, price spike percentage, supplier lead time increase). The model uses five features:

Feature	Type	Extraction Method	Rationale
Event type weight	FLOAT	Lookup table: STRIKE=0.75, LOGISTICS=0.65, GEOPOLITICAL=0.70, FLOOD=0.60, TARIFF=0.55, SHORTAGE=0.70, FINANCIAL=0.65, OTHER=0.30	Base severity varies by disruption category based on historical impact data
Tone score	FLOAT	GDELT AvgTone field or VADER sentiment on non-GDELT sources, normalised to [0,1]	More negative article sentiment correlates with higher actual disruption severity
Named entity density	FLOAT	Count of ORG + GPE entities in article divided by article length in tokens	Higher entity density indicates more specific, targeted event reporting
Source tier	INTEGER (1-4)	Source credibility tier: Reuters/Bloomberg=1, Trade press=2, Regional news=3, Blogs/social=4	Major outlets with editorial standards report more significant events
Corroboration count	INTEGER	Count of distinct source_ids reporting the same event cluster within 24h window	Events covered by multiple independent sources are more severe on average

6.2 Recency Decay Function

Supply chain risk signals have a finite shelf life. A dockworker strike that ended last week has very different relevance than one reported this morning. The recency_weight implements exponential decay with a configurable half-life:

```
recency_weight(t) = exp(-λ × t)
where t = hours elapsed since source_timestamp
and   λ = ln(2) / half_life_hours
```

The default half-life is 14 hours, meaning a signal published 14 hours ago has 50% of the weight of a signal published now. This is configurable per event type — TARIFF events (which have persistent impact) use a 72-hour half-life, while LOGISTICS events (short-term congestion) use an 8-hour half-life.

Corroboration suspension: if an event cluster is being actively reported across multiple sources (corroboration_count increases over time), the decay function is suspended and recency_weight is held at its peak value. This prevents important ongoing events from decaying off the radar while still active.

6.3 Alert Tier Classification

The composite risk score is mapped to one of four alert tiers that control UI presentation and notification behaviour:

Tier	Score Range	UI Presentation	Notification Behaviour	Recommended Action
CRITICAL	0.80 – 1.00	Red banner, pushed to top of dashboard regardless of filter settings	Immediate email notification + in-app push	Contact supplier within 24 hours to assess exposure
HIGH	0.60 – 0.79	Orange card on dashboard, sorted above MEDIUM and LOW	Included in same-day email digest if user has opted in	Review supplier inventory buffer and alternative sourcing options
MEDIUM	0.40 – 0.59	Yellow card, visible in default dashboard view	Included in weekly Monday digest only	Monitor — add to watchlist for trend tracking
LOW	0.00 – 0.39	Grey card, hidden unless user expands LOW filter	Not included in any digest unless user explicitly enables	Background awareness only — no immediate action required

7. Supplier Profile Onboarding System

The supplier profile is the personalisation core of the entire system. Every risk score, every alert, and every graph traversal is anchored to a manufacturer's registered supplier profile. The quality and completeness of the profile directly determines the precision of the alerts — a supplier with a registered country code but no materials_list will receive only geographic alerts, missing commodity-level signals entirely.

7.1 Supplier Profile Schema

Field	Type	Required	How It Is Used
supplier_name	TEXT	Yes	NER entity matching — the supplier name is used as a pattern in the spaCy entity matcher to detect direct mentions in news text
supplier_aliases	TEXT[]	Recommended	Additional name variants (subsidiary names, trade names, former names) added to the NER pattern list
country_code	TEXT (ISO2)	Yes	Geographic overlap scoring — matched against RiskEvent.geographic_scope. Also used to resolve LOCATED_IN link to Country node.
sub_national_region	TEXT	Recommended	Improves geographic precision — province or state code reduces false positives for large countries like China, India, USA
city_or_port	TEXT	Optional	Geocoded to coordinates for spatial proximity to port nodes. Critical for logistics disruption relevance.
materials_list	TEXT[]	Yes	TF-IDF commodity matching — compared against commodity_matches in Gold records to compute commodity_similarity score
criticality_tier	INTEGER (1-3)	Yes	Alert weight multiplier: tier 1 (sole source, no alternative) = ×1.5 weight on composite score. Tier 3 (easily replaceable) = ×0.7.
annual_spend_usd	FLOAT	Optional	Financial exposure quantification shown in alert detail — "estimated annual spend at risk: $2.3M"
lead_time_days	INTEGER	Recommended	Used in impact severity estimation — a supplier with 90-day lead time has higher exposure to logistics disruptions than a 5-day local supplier
alternative_suppliers	INTEGER (count)	Recommended	Used to moderate alert severity — a disruption affecting a sole-source supplier scores higher than one affecting a supplier with three alternatives

7.2 Automatic Supplier Intelligence Enrichment

Upon registration, the platform automatically enriches the supplier profile with data from three sources without requiring any additional input from the user:

- Port proximity resolution — the supplier's city_or_port field is geocoded using offline GeoNames database to latitude/longitude coordinates. The system then runs a PostGIS ST_DWithin query against the ports dataset to find all ports within 500km. These are automatically added as SHIPS_THROUGH link targets in Neo4j, making port-level disruption alerts immediately active.
- Country risk baseline — the Country node for the supplier's country is fetched from Neo4j and its baseline_risk_index property (sourced from the ACLED conflict score and World Bank governance indicators) is applied as a permanent floor modifier to all alerts for this supplier. A supplier in a country with high baseline instability has a minimum composite_risk_score of 0.20 even for low-severity events.

- Commodity price history initialisation — for each item in materials_list, the system fetches the last 90 days of price history from the Commodity nodes in Neo4j and computes the price_volatility_90d metric. High-volatility commodities trigger more sensitive threshold settings for the SHORTAGE event type classifier.

8. Scheduling Architecture and Pipeline Execution

The system operates two scheduling planes that run independently but share the same task queue infrastructure (Celery + Redis).

8.1 Connector Polling Schedule (APScheduler)

Each registered connector is managed by APScheduler with a per-connector schedule stored in the connector config. APScheduler runs in a dedicated FastAPI background task process and maintains a priority queue sorted by next_run_at. On each tick, it fires any connectors whose next_run_at has elapsed, submits them as Celery tasks, and requeues them with next_run_at = now + schedule_seconds.

Connector	Schedule	Celery Queue	Estimated Task Duration	Error Behaviour
GDELT	Every 15 minutes	connectors.high_frequency	8–15 seconds (filtered query)	Exponential backoff: retry after 2min, 8min, 32min. Alert dashboard after 3 failures.
NewsAPI	Every 60 minutes	connectors.standard	3–8 seconds	Retry twice. On third failure mark connector status=degraded.
Freightos	Every 6 hours	connectors.low_frequency	2–5 seconds	Cache last successful response — serve stale data up to 24h before alerting.
World Bank	Daily at 06:00 UTC	connectors.daily	10–30 seconds (full series update)	Daily data — single retry. Failure does not affect news pipeline.
ACLED	Every 4 hours	connectors.standard	5–12 seconds	Rate limited — respect Retry-After header. Queue for deferred execution.
FRED	Daily at 07:00 UTC	connectors.daily	3–8 seconds	Most series are monthly — single retry. Non-critical path.

8.2 NLP Processing Pipeline (Celery Workers)

Bronze-to-Silver and Silver-to-Gold transforms are submitted as Celery tasks to separate queues, allowing CPU-bound NLP tasks to scale horizontally without blocking I/O-bound connector tasks.

Queue: transforms.silver	Handles Bronze → Silver cleaning tasks. One worker per connector type. Low memory requirement — pure text cleaning with pandas. Scales vertically (more CPUs per worker).
Queue: transforms.gold.ner	Handles spaCy NER inference. Workers load the fine-tuned en_core_web_tg model on startup and keep it in memory. One worker handles ~40 articles/minute on a 2-core worker instance.
Queue: transforms.gold.classify	Handles DistilBERT event classification inference. Workers use optimised ONNX Runtime export of the fine-tuned model for CPU inference. Achieves ~60 classifications/minute per worker.
Queue: transforms.gold.score	Handles risk score computation and supplier_matches calculation. Pure Python math — highest throughput queue. One worker processes ~200 score computations/minute.

Queue: ontology.sync	Handles Neo4j node and edge sync after Gold records are produced. Batches up to 500 MERGE operations per Cypher transaction to minimise round-trips.

8.3 End-to-End Latency Budget

From the moment a news article is published to when the corresponding alert appears on the manufacturer's dashboard, the system targets a maximum end-to-end latency of 25 minutes based on the GDELT polling interval. The latency budget is distributed as:

Stage	Latency Budget	Notes
Article published → GDELT indexes it	0–10 minutes	GDELT typically indexes articles within 5 minutes of publication
GDELT polling interval	0–15 minutes	APScheduler fires GDELT connector every 15 minutes
Bronze write + Silver transform	< 30 seconds	Cleaning pipeline is I/O and CPU-light
NER inference (spaCy)	< 60 seconds	Queued immediately after Silver write
DistilBERT classification	< 90 seconds	Queue depth determines actual latency; single article inference ~15ms
Risk scoring + supplier matching	< 30 seconds	Math-only — no ML inference
Ontology mapper (Neo4j MERGE)	< 60 seconds	Batched every 30 seconds
Dashboard WebSocket push	< 5 seconds	React dashboard polls /alerts endpoint every 30 seconds
TOTAL worst case	~26 minutes	Dominated by GDELT polling interval

9. PostgreSQL Database Schema

9.1 Core Application Tables

Table	Purpose	Key Columns
manufacturers	Registered platform users	manufacturer_id (UUID PK), company_name, industry_naics_code, hq_country, risk_tolerance (LOW/MEDIUM/HIGH), created_at
suppliers	User-registered supplier profiles	supplier_id (UUID PK), manufacturer_id (FK), supplier_name, aliases (TEXT[]), country_code, sub_national_region, city, geom (GEOMETRY Point), materials_list (TEXT[]), criticality_tier, lead_time_days, alternative_count, annual_spend_usd
alert_log	All generated alerts with full audit trail	alert_id (UUID PK), manufacturer_id (FK), supplier_id (FK), event_id (FK), alert_tier, composite_risk_score, exposure_path (JSONB), created_at, acknowledged_at, dismissed_at
user_preferences	Per-manufacturer alert configuration	manufacturer_id (FK PK), min_score_threshold, score_weights (JSONB), digest_enabled, digest_day, notification_email, suppress_event_types (TEXT[])

9.2 Pipeline Tables

Table	Tier	Key Columns
raw_data	Bronze	id (BIGSERIAL), connector_id, source_type, payload (JSONB), row_hash (UNIQUE), ingest_timestamp, source_timestamp, geographic_scope (TEXT[]), commodity_refs (TEXT[])
silver_news	Silver	article_id (TEXT PK), source_connector, headline, body_text, source_name, source_credibility, published_at, geographic_scope (TEXT[]), raw_id (FK), silver_computed_at
silver_freight	Silver	index_id (TEXT PK), route_code, route_name, index_value (DECIMAL), change_pct (DECIMAL), statistical_anomaly (BOOLEAN), recorded_at, raw_id (FK)
silver_commodities	Silver	price_id (TEXT PK), commodity_id, price_value_usd (DECIMAL), change_pct, statistical_anomaly, unit, period_date, raw_id (FK)
silver_conflict	Silver	event_id (TEXT PK), event_type, country_code, admin1, latitude, longitude, geom (GEOMETRY Point), fatalities, event_date, raw_id (FK)
gold_risk_events	Gold	event_id (TEXT PK), article_id (FK), event_type, event_type_confidence, event_severity, extracted_entities (JSONB), supplier_matches (JSONB), commodity_matches (TEXT[]), port_matches (TEXT[]), route_matches (TEXT[]), recency_weight, source_credibility, composite_risk_score, gold_computed_at
lineage	Audit	id, output_table, output_id, input_table, input_id, transform_name, computed_at — full provenance chain from Gold back to Bronze

9.3 Ontology Registry Tables

Table	Purpose
object_type_registry	Runtime Object Type definitions: Supplier, Port, Commodity, ShipmentRoute, RiskEvent, Country, Manufacturer. Zero hardcoding — all seven types are rows in this table.
link_type_registry	Runtime Link Type definitions with resolver strategies. All 12 link types described in Section 5.2 are rows here. The graph engine reads these at runtime to know which relationships to build.
ontology_source_mappings	Maps each dataset or connector to its Object Type: source_reader (direct_table or json_payload), property_mappings (JSONB), incremental_key, sync_frequency_s.
ontology_suggestions	Auto-generated suggestions from the Auto-Builder when new datasets are ingested. For supplier registrations, suggestions are auto-accepted at confidence=1.0.

10. Frontend Architecture and Notification System

10.1 React Dashboard — Key Views

View	React Component	Data Source	Key Technical Details
Risk Heatmap	SupplierRiskMap	GET /alerts?manufacturer_id=X&min_score=0.4	Recharts Treemap with cell size proportional to annual_spend_usd and cell colour mapped to highest composite_risk_score for that supplier. Clicking a cell opens the supplier's alert list.
Alert Feed	AlertFeed	GET /alerts?status=active&sort=composite_risk_score	Virtualized list (react-window) for performance with large alert volumes. Severity badge colour-coded by tier. Expandable to show exposure_path explanation.
Supplier Profiles	SupplierManager	GET /suppliers, POST /suppliers, PATCH /suppliers/:id	Form-based CRUD. Geocoding of city field happens server-side on POST — client receives latitude/longitude back. Real-time enrichment status shown (port matches found, country risk index loaded).
Event Explorer	EventExplorer	GET /events?types=STRIKE,LOGISTICS&date_from=X	Filterable table of all Gold-tier events regardless of supplier matching. Allows analysts to explore signals not yet tied to their supplier profile.
Route Monitor	FreightDashboard	GET /freight-indices	Line charts (Recharts) of FBX route indices over 90 days. Statistical anomaly flags displayed as red dots on the time series. Threshold crossing alerts configurable.
Weekly Digest Preview	DigestPreview	GET /digest/preview?manufacturer_id=X	Server-rendered HTML preview of the upcoming Monday email digest. Shows exactly what the email will contain before it is sent.

10.2 SendGrid Email Digest — Technical Implementation

The weekly digest is a server-rendered HTML email generated every Monday at 07:00 UTC by a Celery beat task. For each manufacturer with digest_enabled=True, the system:

- Queries all alerts created since the previous Monday with composite_risk_score >= 0.40, grouped by supplier and sorted by peak risk score
- Fetches the top 3 contributing events per supplier using the alert_log.exposure_path JSONB field
- Renders a Jinja2 HTML template using the alert data — inline CSS only (no external stylesheets, email client compatibility)
- Attaches a commodity price summary table from the last 7 days of silver_commodities data for commodities in the manufacturer's materials_list
- Calls SendGrid v3 /mail/send API endpoint with the rendered HTML and a plain-text fallback generated by stripping HTML tags
- Stores digest_sent_at and sendgrid_message_id in the manufacturers table for delivery tracking

Personalisation beyond supplier filtering: the digest reorders sections based on the manufacturer's industry_naics_code. A food manufacturer sees agricultural commodity signals first; an electronics manufacturer

sees semiconductor and rare earth signals first. This reordering is driven by a static industry-to-commodity priority mapping table.

11. End-to-End Data Flow Summary

The following describes the complete journey of a supply chain disruption signal from first detection to manufacturer alert, with every technical step named explicitly.

COMPLETE SIGNAL FLOW — PORT STRIKE EXAMPLE

STEP 1 — DETECTION

APScheduler fires GDELT connector task every 15 minutes.

httpx async GET to GDELT GKG v2 endpoint with THEMES filter including LABOR_DISPUTE, TRANSPORT.

Response decoded from tab-delimited format to pandas DataFrame.

Each row wrapped in Canonical Payload with source_type="news", geographic_scope extracted from ActionGeo fields.

SHA-256 row_hash computed. Duplicate check against raw_data.row_hash. New records appended to raw_data (Bronze).

STEP 2 — SILVER TRANSFORM

Celery task submitted to transforms.silver queue.

HTML entity decoding, URL canonicalisation, language detection (langdetect).

Publication date parsed to UTC with dateutil.parser.

MinHash LSH near-duplicate detection (datasketch library) against last 24h of silver_news.

Clean record inserted into silver_news table.

STEP 3 — NLP PIPELINE (GOLD TRANSFORM)

Celery task submitted to transforms.gold.ner queue.

spaCy en_core_web_tg pipeline runs on headline + body_text.

Extracted entities: ORG="Colombo Port Authority", GPE="Sri Lanka", GPE="Colombo".

Colombo geocoded to LK + port proximity → UNLOCODE LKCMB matched.

Celery task submitted to transforms.gold.classify queue.

DistilBERT classifier returns: STRIKE (confidence=0.91).

Celery task submitted to transforms.gold.score queue.

Recency weight computed: $\exp(-0.05 \times 2h) = 0.905$.

Source credibility: Reuters tier 1 → 0.90.

Supplier matching: TF-IDF + geographic overlap run against all manufacturers' supplier profiles.

Three suppliers matched with relevance > 0.25: supplier_A (0.87), supplier_B (0.61), supplier_C (0.44).

Composite risk scores: 0.81 (CRITICAL), 0.65 (HIGH), 0.47 (MEDIUM).

Gold record inserted into gold_risk_events. Lineage record written.

STEP 4 — ONTOLOGY SYNC

Celery task submitted to ontology.sync queue.

Neo4j MERGE: RiskEvent node created with all Gold properties.

AFFECTS_PORT edge: RiskEvent → Port(LKCMB) via port_matches.

AFFECTS_COUNTRY edge: RiskEvent → Country(LK) via geographic_scope.

EXPOSES edges: RiskEvent → Supplier(A), Supplier(B), Supplier(C) via supplier_matches.

STEP 5 — ALERT GENERATION

Alert engine queries gold_risk_events for records with composite_risk_score > manufacturer threshold.

For each match, exposure_path resolved via Cypher traversal.

Alert records written to alert_log: tier=CRITICAL for supplier_A, tier=HIGH for supplier_B.

React dashboard receives new alerts on next 30-second poll.

If tier=CRITICAL and manufacturer.risk_tolerance=HIGH: immediate SendGrid email triggered.

TOTAL TIME FROM PUBLICATION TO DASHBOARD: approximately 18–26 minutes.

DESIGN PRINCIPLES SUMMARY

Every data record is traceable — from dashboard alert back to the exact source article via Bronze lineage.

Every alert is explainable — exposure_path shows exactly why a supplier is flagged and through what chain of relationships.

Every schema is runtime-defined — Object Types, Link Types, and source mappings are database rows, not code.

Every NLP output is typed — raw text becomes structured Gold records before any business logic runs on it.

Every threshold is configurable — risk weights, severity decay, score floors, and digest settings are per-manufacturer.